

C — Technical questions for the Perforce review call

*Send these in advance of the call (attached to the email in **A**, or as a follow-up the day before) so engineering can come prepared. Grouped from "blocking" to "nice to know."*

Group 1 — Correctness of move discovery (blocking)

These determine whether the tool's output is *complete*. Wrong answers here mean Amdocs's p4 zip could still fail.

1. **Does p4 filelog -ztag //depot/... always surface both halves of a move/add ↔ move/delete pair, even when the two halves live in different sub-paths?** *Why we ask:* we currently scan the full depot to sidestep partial visibility. If a path filter would suppress one side, we need to know so we can adjust the discovery query.
2. **For a move/add revision, is movedFile guaranteed to point at the move/delete counterpart, or can it be empty / pre-truncated by server housekeeping (e.g. p4 obliterate, p4 archive)?** *Why we ask:* if movedFile can be NULL on either side, we need a secondary lookup (e.g. integration records).
3. **How are move/add records affected by p4 obliterate?** Specifically: does obliterating one half leave a dangling movedFile reference on the other half? And does p4 zip then complain about the obliterated half being "outside the view"?
4. **Cross-changelist move chains** — if a file moves A → B in change 100, then B → C in change 200, does Amdocs need *both* hops in the remote spec for p4 zip to succeed across the full history? Should we do a transitive-closure pass over movedFile?
5. **Branch-then-move and integration-then-move sequences** — are these reported as move/* actions, or as branch/integrate+add/delete? If they reach p4 zip as integration records, is the same "partial action" error class possible, and should the spec cover them too?

Group 2 — Server-version & flag dependencies

6. **Which server versions are known to emit different movedFile semantics?** We're targeting whatever Amdocs is running today; please confirm the version range for which our discovery query is reliable.
7. **Server configurables that affect this** — are there any (filesys.depot.move, dm.shelve.promote, dm.protects.allow.admin, etc.) that would change either the filelog output or p4 zip's acceptance criteria?
8. **Streams vs. classic depots** — does the Amdocs depot mix the two, and does p4 zip apply identical view-completeness checks in both cases?

Group 3 — Performance & scale

9. **Is there a bulk query more efficient than filelog -c <range> //depot/... for ~700k files × decades of history?** We considered p4 changes + p4 -ztag describe -s as a per-changelist alternative; would that scale better, or worse?
10. **Server-load guidance** — what is a safe --workers ceiling for parallel filelog calls against a production server, and is there an off-hours flag / replica we should target?

11. **p4 -ztag vs. p4 -F formatting** — any reason to prefer one for large-output piping? We're currently relying on -ztag parsing via P4Python.

Group 4 — Interaction with the official enhancement

12. **What's the rough timeline for the p4 zip enhancement** that auto-handles move pairs? Even an unofficial quarter target helps us decide whether this prototype is bridge code or longer-lived.
13. **Will the enhancement change the wire format of remote specs**, or deprecate view lines in favour of something else? We'd like to avoid generating a spec that breaks on the next release.
14. **Would Perforce prefer that the prototype** (a) be retired the moment the enhancement ships, (b) be carried forward as a community tool, or (c) be folded into a Perforce-owned location? We'll align with whatever you prefer.

Group 5 — Process / customer-facing

15. **Are you comfortable with us sharing the tool with Amdocs?** With or without Perforce endorsement is fine — we just want to be transparent before doing so.
16. **If something goes wrong on Amdocs's side**, what's the right support pathway? Continue through the existing case, open a new one, or route through our reseller channel?
17. **Is there a Perforce engineer** we should add to the customer-side call when we eventually demo this to Amdocs, so they have direct vendor presence?

Suggested call structure (20 min)

Time	Topic
0–3	Recap of the problem + Francesco's original workaround
3–8	Live demo: scan → build-spec → zip against sample data
8–15	Walk Group 1 questions, capture engineering's answers
15–18	Group 2–3 if time permits, otherwise async follow-up
18–20	Decide: share with Amdocs? co-brand? hold?

Maintained alongside the source: any answer Perforce gives that changes the implementation should be reflected in [../.. /src/p4_move_zip_fixer/](#) and re-tested before customer delivery.